

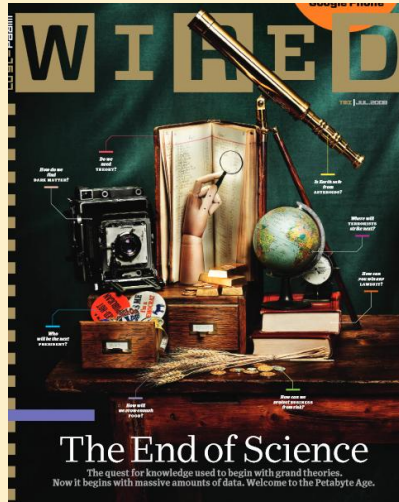
Big Data Analytics

Intro

Data storage and distributed processing

Mihaela Elena Breabăn

© FII 2025-2026



“The End of Theory: The Data Deluge Makes the Scientific Method Obsolete”

The quest for knowledge used to begin with grand theories. Now it begins with massive amounts of data. Welcome to the Petabyte Age!

Chris Anderson,
in Wired Magazine, 2008

<https://www.wired.com/2008/06/pb-theory/>



The Unreasonable Effectiveness of Data

Alon Halevy, Peter Norvig, and Fernando Pereira, Google

“We don’t have better algorithms than anyone else. We just have more data.”

Peter Norvig (2011)

<https://www.computer.org/csdl/magazine/ex/2009/02/mex2009020008/13rUY0HYOb>

More data usually beats better algorithms

I teach a [class on Data Mining](#) at Stanford. Students in my class do a project that does some non-trivial data mining. Many students take their hand at the [Netflix Challenge](#): to design a movie recommendation algorithm that does better than the one developed by Netflix.

“...adding more, independent data usually beats out designing ever-better algorithms to analyze an existing data set. I’m often surprised that many people in the business, and even in academia, don’t realize this.”

Anand Rajaraman (2008)

<http://anand.typepad.com/datawocky/2008/03/more-data-usual.html>

Data growth -> data mining

- ▶ Lots of data is being collected and warehoused
 - ▶ purchases at department/grocery stores
 - ▶ Bank/Credit Card transactions
 - ▶ Web data, e-commerce
 - ▶ Lots of web pages
 - ▶ Hundreds of millions tweets per day
 - ▶ Billions of Google queries per day
 - ▶ A billion Facebook users
 - ▶ Millions of servers, petabytes of data
- ▶ Competitive Pressure is Strong
 - ▶ Provide better, customized services (e.g. in Customer Relationship Management)



Why not just sampling?

- ▶ Must think about...
 - ▶ Is it the right data?
 - ▶ What about very rare events? (anomalies such as fraud, supply chain disruption,...)
- ▶ ...and must remember: *The larger the data sample, the more accurate the statistics!*

When the big data software technology is mature enough, sampling is not good enough



Agenda

- ▶ Big Data - storage
- ▶ Big Data - processing





STORAGE

Database Management Systems

- ▶ A complex system made of
 - ▶ Hardware
 - ▶ Software
 - ▶ Data
 - ▶ Users
 - ▶ Provides support for storing data, and efficient data manipulation
 - ▶ Features:
 - ▶ Security
 - ▶ Controlled database access
 - ▶ Data storing, retrieval, update
 - ▶ Integrity
 - ▶ Transaction control
 - ▶ Concurrent control
 - ▶ Data backup and recovery
 - ▶ Catalog (data dictionary)
-



Data models

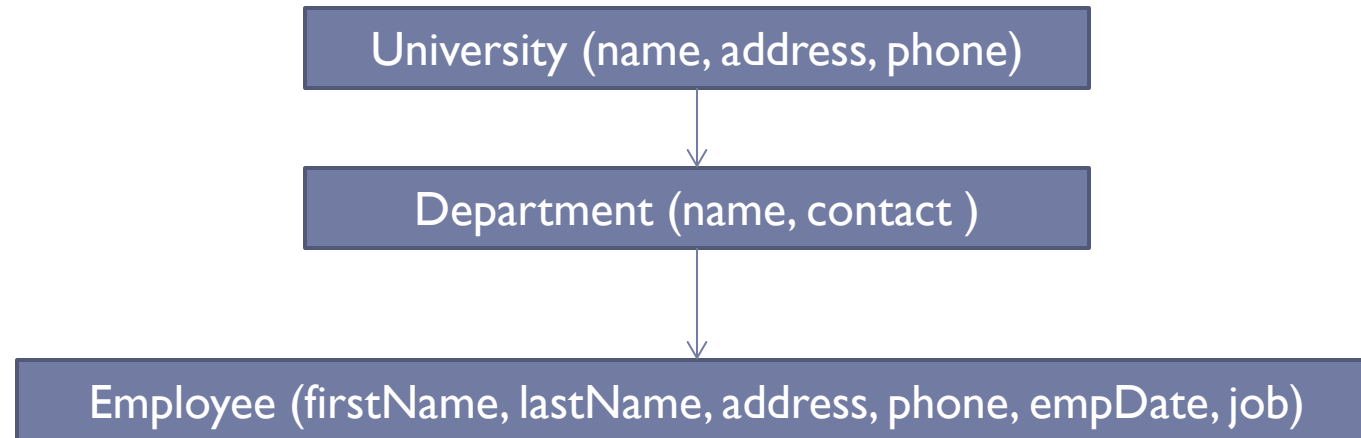
- ▶ Hierarchical model (IBM's IMS, late '60s)
- ▶ Network Model (CODASYL 1969-1971)
- ▶ Relational model (Codd, '70s)
- ▶ Object-relational model ('90s)
- ▶ XML DBs (2000)
- ▶ NoSQL family



The First Decade B.C.*

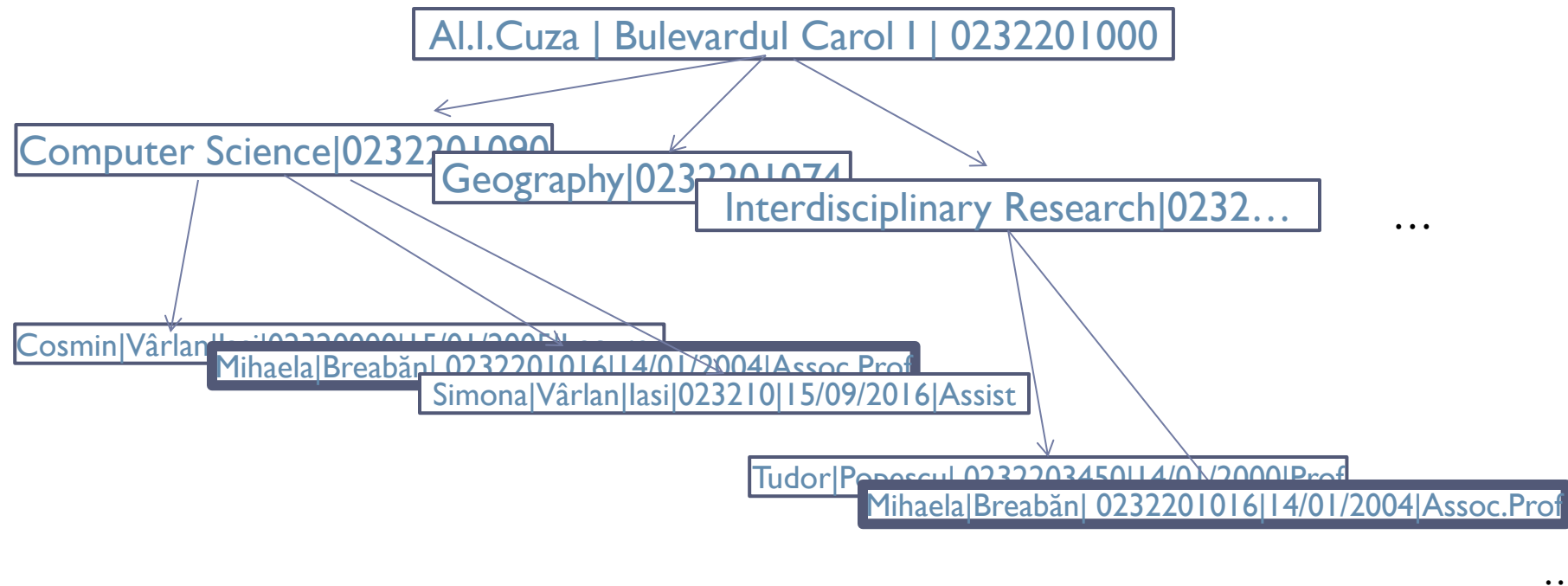
Hierarchical and network DB systems

- ▶ Data organized as records which are stored as nodes in graphs
- ▶ *Navigational APIs*
 - ▶ Programmers had to (carefully!) scan or search for records, follow parent/child structures or pointers, and maintain code when anything physical changed



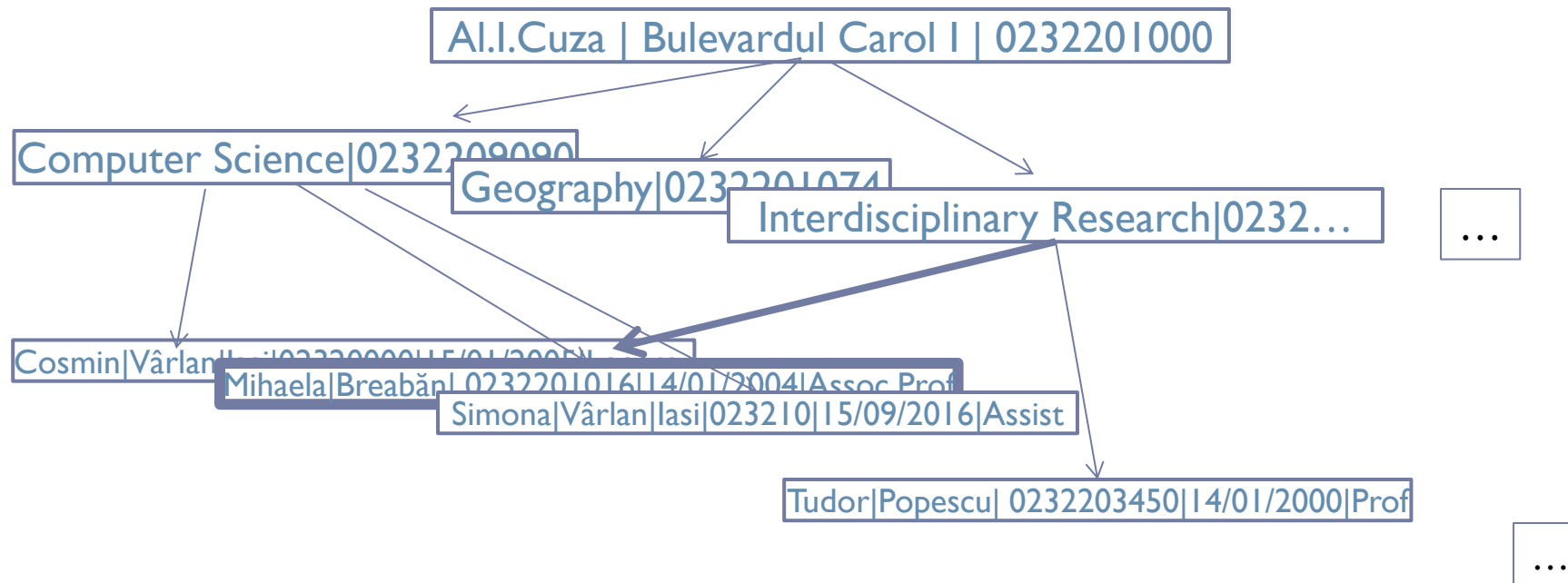
Hierarchical model (IBM's IMS, '60s)

- ▶ Able to model only one-to-many relationships



Network model (Charles Bachman - CODASYL 1969-1971)

- ▶ Extends the hierarchical model by allowing many-to-many relationships



Relational model

(Edgar Frank Codd – '70)

IBM's System R (SEQUEL)
Berkley's Ingres
Oracle, Postgres, SQLServer,
MySQL...

- ▶ **Everything is (logical) rows and columns!**
- ▶ **Flat structures: columns are atomic (1NF)**
- ▶ **Data INDEPENDENCE!**
- ▶ **Components:**
 - ▶ *Tables* as data structures
 - ▶ Constraints on data stored in tables
 - ▶ Logical relationships between tables (via Keys)
 - ▶ Methods to build new tables from existing ones (operators in relational algebra)

1. Information Rule
2. Guaranteed Access Rule
3. Comprehensive Data Sub-language Rule
4. View Update Rule
5. High Level Insert, Update and Delete
6. Physical Data Independence
7. Logical Data Independence
8. Integrity Independence
9. Non Subversion Rule
10. Systematic Treatment of Null Values
11. Database Description Rule
12. Distribution Independence

Relational model (cont.)

Departments

name	contact	deptID
Computer Science	0232201090	1
Geography	0232201074	2
Interdisciplinary Research	0232201102	3

Employees

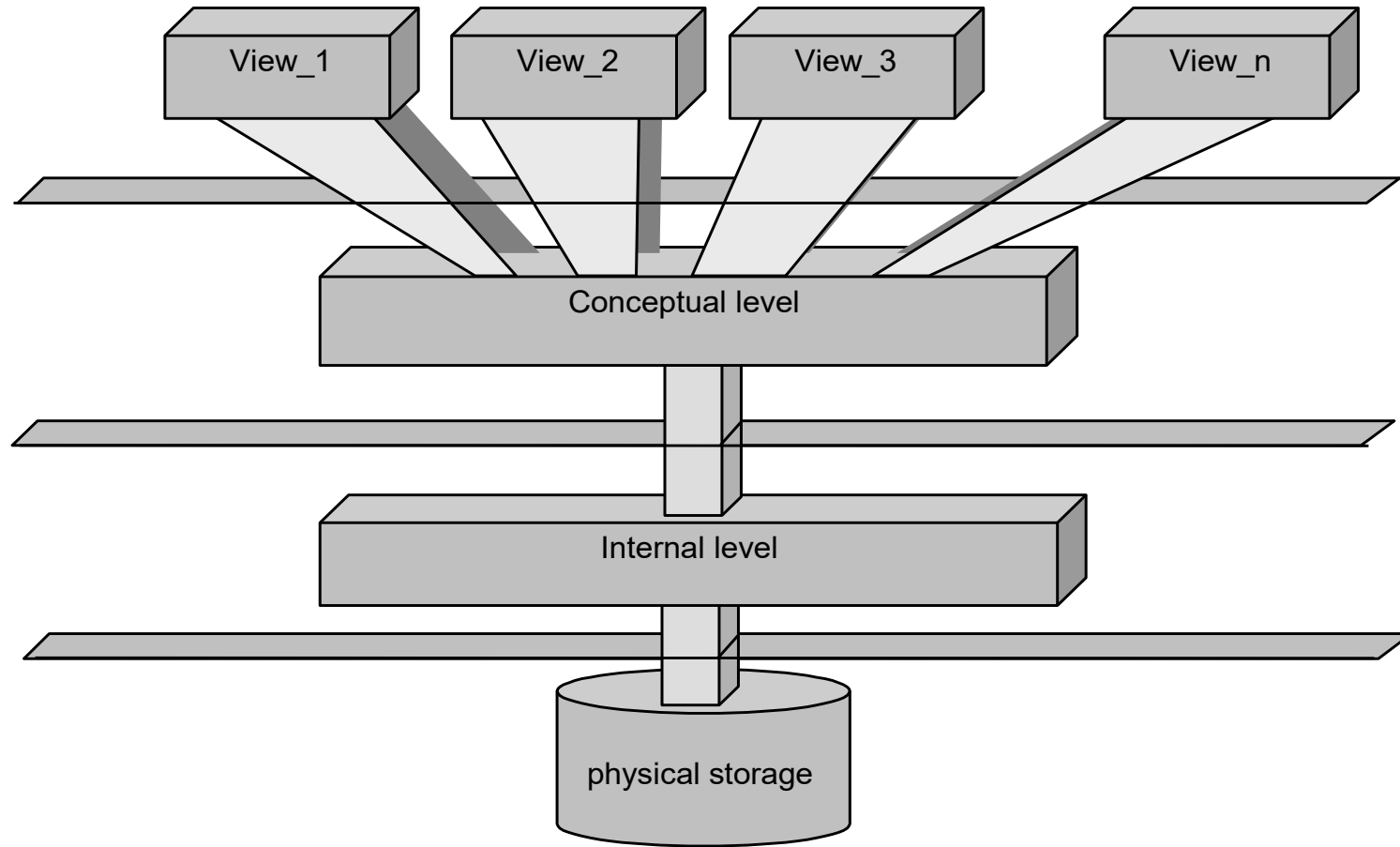
firstName	lastName	address	phone	empDate	job	empID
Cosmin	Vârlan	Iasi	0232100000	15/01/2005	Lecturer	11
Mihaela	Breabăn	Iasi	0232201016	14/01/2004	Assoc.Prof	22
Simona	Vârlan	Iasi	0232100000	15/09/2016	Assist.	33
Tudor	Popescu	Iasi	0232203450	14/01/2000	Professor	44

Dept_Emp

deptID	empID
1	11
1	22
1	33
3	22
3	44

The relational model (reviewed)

ANSI-SPARC architecture



Object-relational model (’90s)

Postgres, Starburst,
UniSQL, Illustra, DB2,
Oracle

- ▶ Related to OO programming languages
 - ▶ Methods (“behavior”) as well as data in the DBMS
 - ▶ *User-defined functions (UDTs/UDFs) & aggregates, nested tables*

Employees

Name		address	phone	empDate	job	empID
Cosmin	Vârlan	Iasi	0232200000	15/01/2005	Lecturer	11
Mihaela	Breabăn	Iasi	0232201016	14/01/2004	Assoc.Prof	22
Simona	Vârlan	Iasi	0232100000	15/09/2016	Assist.	33
Tudor	Popescu	Iasi	0232203450	14/01/2000	Professor	44

Departments

name	contact	Employees		
Computer Science	0232201090	11	22	33
Geography	0232201074			
Interdisciplinary Research	0232/201102	22		44

XML databases (2000)

*Natix, Timber, Ipedo,
MarkLogic, BaseX; DB2,
Oracle, SQL Server*

- ▶ Flexible data model
 - ▶ Origins in document markup (SGML)
 - ▶ Nested data
 - ▶ Schema variety/optionality
- ▶ New declarative query language (XQuery)
 - ▶ Designed both for querying and transformation
 - ▶ Early standardization effort (W3C)

```
<Department id=1>
  <name>Computer Science</name>
  <contact>0232201090</contact>
  <employees>
    <employee> 11</employee>
    <employee> 22</employee>
    <employee> 33</employee>
  </employees>
...
```

```
<Employee id=11>
  <firstName>Mihaela</firstName>
  <lastName>Breabăn</lastName>
  <phone>0232201016</phone>
  <address>Iasi</address>
...
```

NoSQL databases

...current investments...

- ▶ The movement defined by “what it’s not”
 - ▶ Triggered by the development of web and the decrease of storage costs
 - ▶ Developed in the *distributed systems* community
 - ▶ Encouraged by the development of artificial intelligence and cloud computing
 - ▶ From “how to store it?” to “how to use it?” – optimized for developer productivity
-
- ▶ SQL is about traditional relational DBMS and not the SQL language
 - ▶ NoSQL is not really about SQL, but developing data management systems that are not relational.

NoSQL

The aim

- ▶ NoSQL databases are useful for problems with:
 - ▶ Variable data
 - ▶ semi-structured, evolving, or has no schema
 - ▶ Massive data
 - ▶ petabytes of data from new applications (web analysis, sensors, social graphs)
 - ▶ Parallelism
 - ▶ large data requires architectures to handle massive parallelism, scalability, and reliability
 - ▶ Horizontal scaling
 - ▶ Simpler queries
 - ▶ may not need full SQL expressiveness
 - ▶ Relaxed consistency:
 - ▶ more tolerant of errors, delays, or inconsistent results ("eventual consistency")
 - ▶ Easier/cheaper
 - ▶ less initial cost to get start



NoSQL systems

Types

😊 Two MySQL DBAs walk to a NoSQL bar. They left disappointed: they couldn't find any tables!

▶ Distributed File Systems

- ▶ No data model, data stored in files
- ▶ User provides specific functions
- ▶ System provides data processing “glue”, fault-tolerance, scalability, designed for large scale analysis

▶ Key-value stores

- ▶ ideal for retrieving specific data records from a large set of data
- ▶ Data model: (key, value) pairs
- ▶ Operations: Insert(key,value), Fetch(key), Update(key), Delete(key)
- ▶ Examples: Redis, Amazon Dynamo, Voldemort

▶ Document stores

- ▶ Data model: similar to key-value stores except value is a document in some form (e.g. JSON, xml), also Fetch based on document contents
- ▶ Examples: MongoDB , CouchDB, SimpleDB

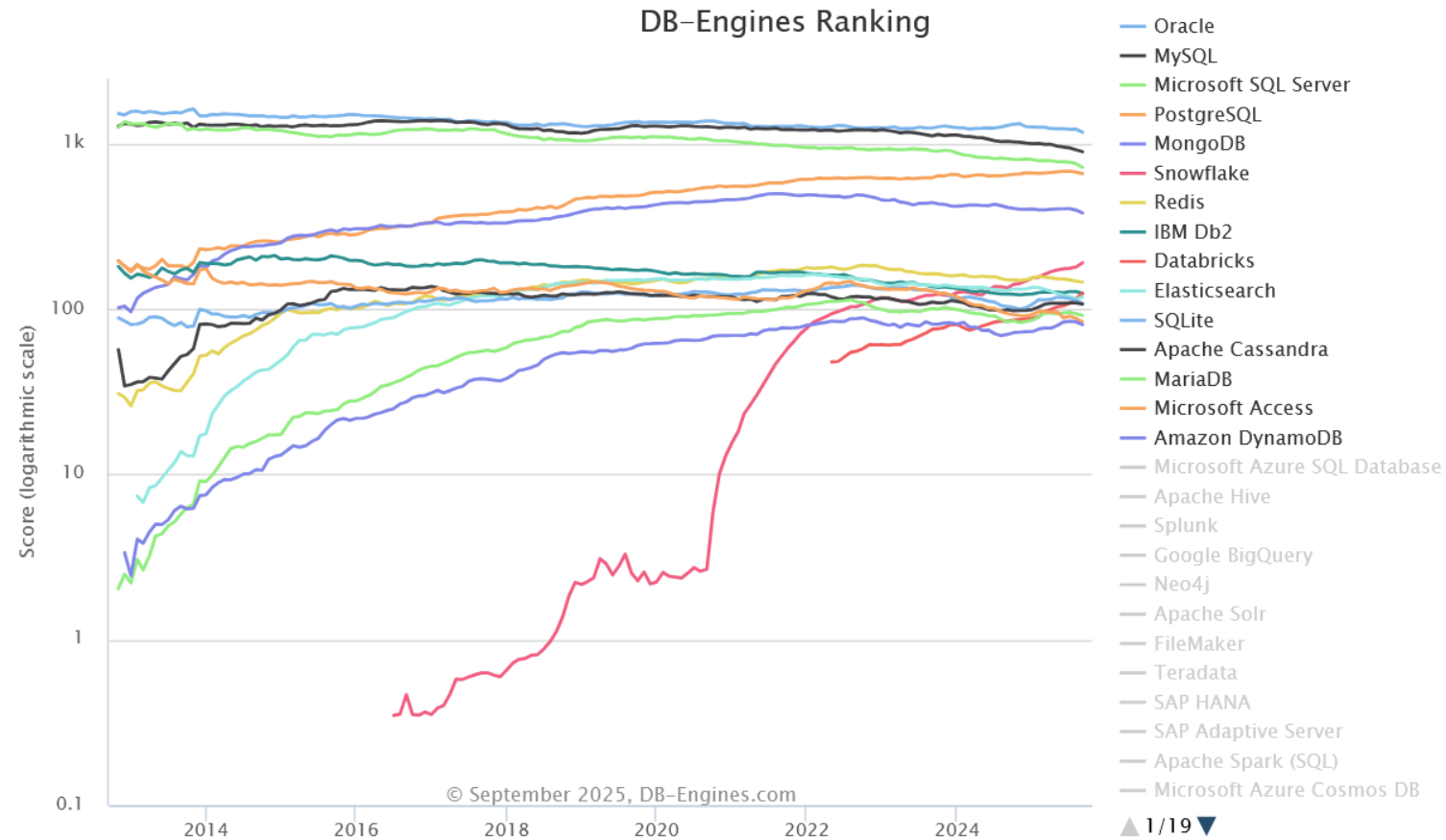
▶ Column-Family Stores

- ▶ Data organized as column families, useful for analytical queries
- ▶ Examples: BigTable, Apache Cassandra, HBase, ScyllaDB

▶ Graph databases

- ▶ Data model: nodes and edges; Nodes may have properties (including ID); Edges may have labels or roles

DB-Engines Popularity trend



https://db-engines.com/en/ranking_trend

Are RDBMSs alone the answer to all data management/analysis problems?

- ▶ Relational databases are the dominant form of database and apply to many data management problems.
- ▶ Features:
 - ▶ Convenient :
 - ▶ Simple data model
 - ▶ Declarative query language
 - ▶ Transaction guarantees
 - ▶ Multi-user
 - ▶ Safe
 - ▶ Persistent
 - ▶ Reliable
 - ▶ Massive
 - ▶ Efficient
- ▶ Not for big data analytics but they are clearly a mainstay
- ▶ Some features not appropriate for certain problems:
 1. Fixed schemas - the schemas must be defined ahead, changes are difficult
 2. Complicated queries
 3. Transaction overhead
 4. Scalability



Example

Web log analysis

- ▶ Each record: UserID, URL, timestamp, additional-info
- ▶ Separate records: UserID, name, age, gender, ...
- ▶ Task: Load into database system
 - ▶ Data cleaning, data extraction, schema specification
 - ▶ Processing may involve cleaning and data extraction
- ▶ Task: Find all records for...
 - ▶ Given UserID
 - ▶ Given URL
 - ▶ Given timestamp
 - ▶ Certain construct appearing in additional-info
 - ▶ Highly parallelizable!
- ▶ Task: Find average age of user accessing given URL
 - ▶ relaxed consistency

Challenges in big data analytics

- ▶ Variety of data types
- ▶ Compute-intensive deep analytics that touch large amounts of data
- ▶ Parallelism
- ▶ Fault tolerance at scale
- ▶ Price-performance challenge



CAP theorem

- ▶ The CAP theorem (proposed by Eric Brewer) states that in a data system, if considering the following three properties:
 - ▶ Consistency (in a system containing replicas, these must be all up-to-date before they can be accessed)
 - ▶ Availability (for a distributed system to be continuously available, every request received by a node in the system must result in a response)
 - ▶ Partitions

you can have at most two of the three properties at a time.

- ▶ Since scaling out requires partitioning, many NoSQL systems sacrifice consistency for availability.



Eventual Consistency

- ▶ Updates will eventually propagate through all nodes in the system which will then be consistent.
- ▶ Criticism: does not make safety guarantees: an eventually consistent system can return any value before it converges.
- ▶ **BASE**
 - ▶ Basically Available
 - ▶ Soft state
 - ▶ Eventual consistency
- ▶ Different consistency properties than ACID.



NoSQL Systems - concluded

Alternative to traditional relational DBMS

- + Flexible schema
- + Quicker/cheaper to set up
- + Massive scalability
- + Relaxed consistency -> higher performance & availability
- No declarative query language -> more programming
- Relaxed consistency -> fewer guarantees



Data Warehouses vs. Data Lakes

 **Business Intelligence**  **SQL Analytics**

 **Governance and Security**
Table ACLs



Data warehouse

(structured tables, requires ETL)

- + BI and reporting
- + unifies disparate systems
- poor support for unstructured data, AI, streaming
- closed and proprietary formats
- expensive to scale

 **Data Science & ML**  **Data Streaming**

 **Governance and Security**
Files and Blobs



Data lake

(unstructured files, brute data)

- + stores any kind of data
- + inexpensive storage
- + good starting point
- complex to set up
- poor BI performance
- can become data swamps



PROCESSING

About data volume and processing power...

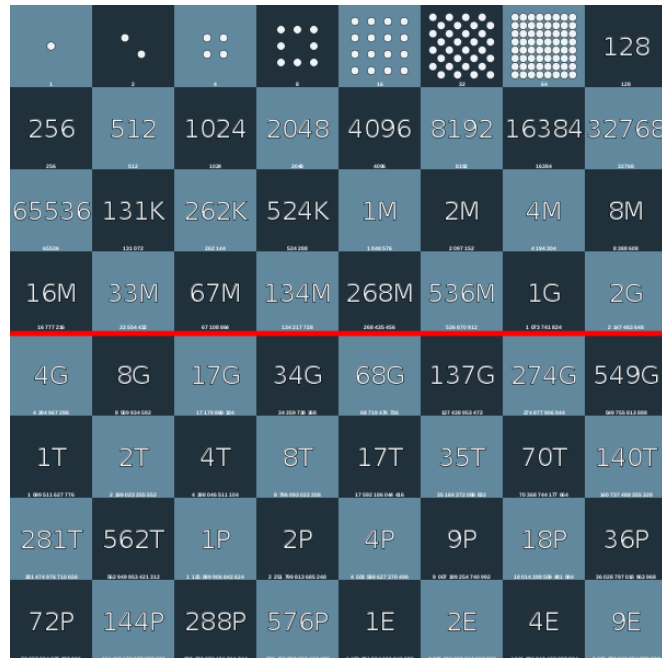
A story: the second half of the chessboard

- There are different stories about the invention of chess. One of them includes the geometric progression problem. Its earliest written record is contained in the [Shahnameh](#), an epic poem written by the Persian poet [Ferdowsi](#) between c. 977 and 1010 CE.

When the creator of the game of chess (in some tellings an [ancient Indian Brahmin](#)^{[1][2]} mathematician named [Sessa or Sissa](#)) showed his invention to the ruler of the country, the ruler was so pleased that he gave the inventor the right to name his prize for the invention. The man, who was very clever, asked the king this: that for the first square of the chess board, he would receive one grain of wheat (in some tellings, rice), two for the second one, four on the third one, and so forth, doubling the amount each time. The ruler, arithmetically unaware, quickly accepted the inventor's offer, even getting offended by his perceived notion that the inventor was asking for such a low price, and ordered the treasurer to count and hand over the wheat to the inventor. However, when the treasurer took more than a week to calculate the amount of wheat, the ruler asked him for a reason for his tardiness. The treasurer then gave him the result of the calculation, and explained that it would take more than all the assets of the kingdom to give the inventor the reward. The story ends with the inventor becoming the new king. (In other variations of the story the king punishes the inventor.)

- Speaking in numbers:
 - ▶ The number of grains of rice on the **first half** of the chessboard is $4,294,967,295$ ($2^{32} - 1$), or about 100,000 kg of rice (assuming 25 mg as the mass of one grain of rice).
 - ▶ The number of grains of rice on the **second half** of the chessboard is $2^{32} + 2^{33} + 2^{34} \dots + 2^{63}$, for a total of $2^{64} - 2^{32}$. The first square of the second half alone contains more grains than the entire first half.
 - ▶ On the entire chessboard there would be $2^{64} - 1 = 18,446,744,073,709,551,615$ grains of rice, weighing 461,168,602,000 [metric tons](#), which would be a heap of rice larger than [Mount Everest](#). This is around 1,000 times the global production of rice in 2010 (464,000,000 metric tons).

Behind the story

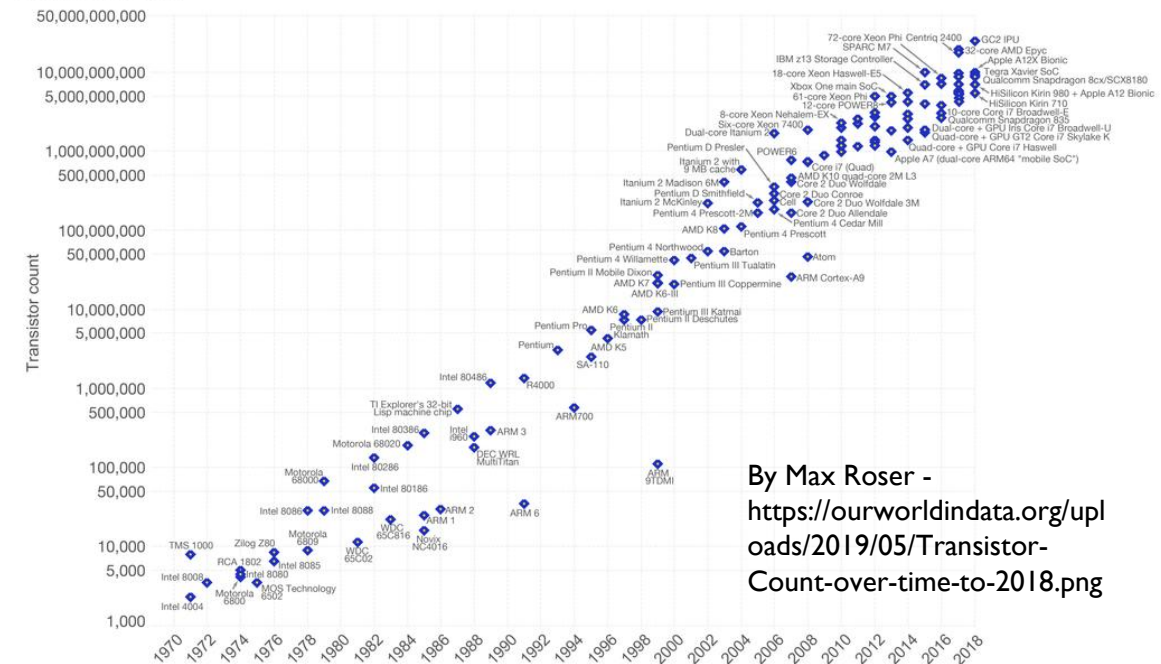


Kryder's Law (2005):
the density of information
doubles roughly every 13 months

Moore's Law – The number of transistors on integrated circuit chips (1971-2018)

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are linked to Moore's law.

Our World
in Data

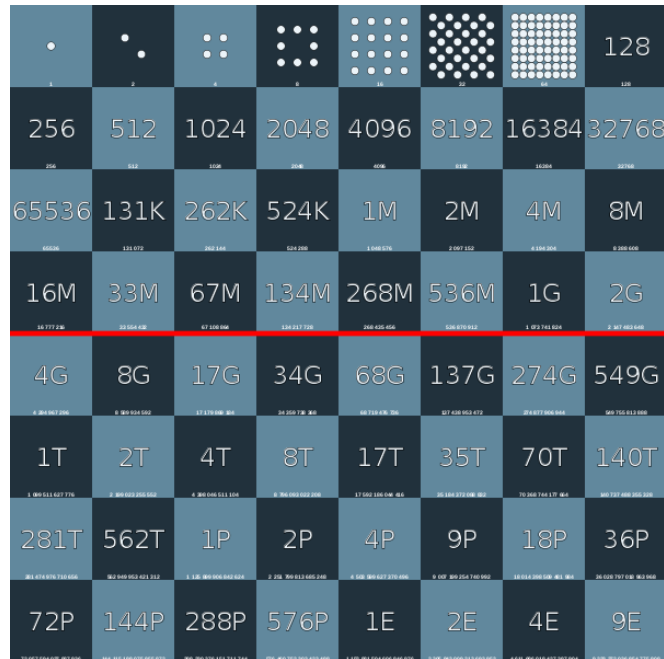


Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldInData.org. There you find more visualizations and research on this topic.

Licensed under CC-BY-SA by the author Max Roser.

Moore's Law (1975):
the number of transistors on integrated
circuits doubles approximately every
two years

Behind the story

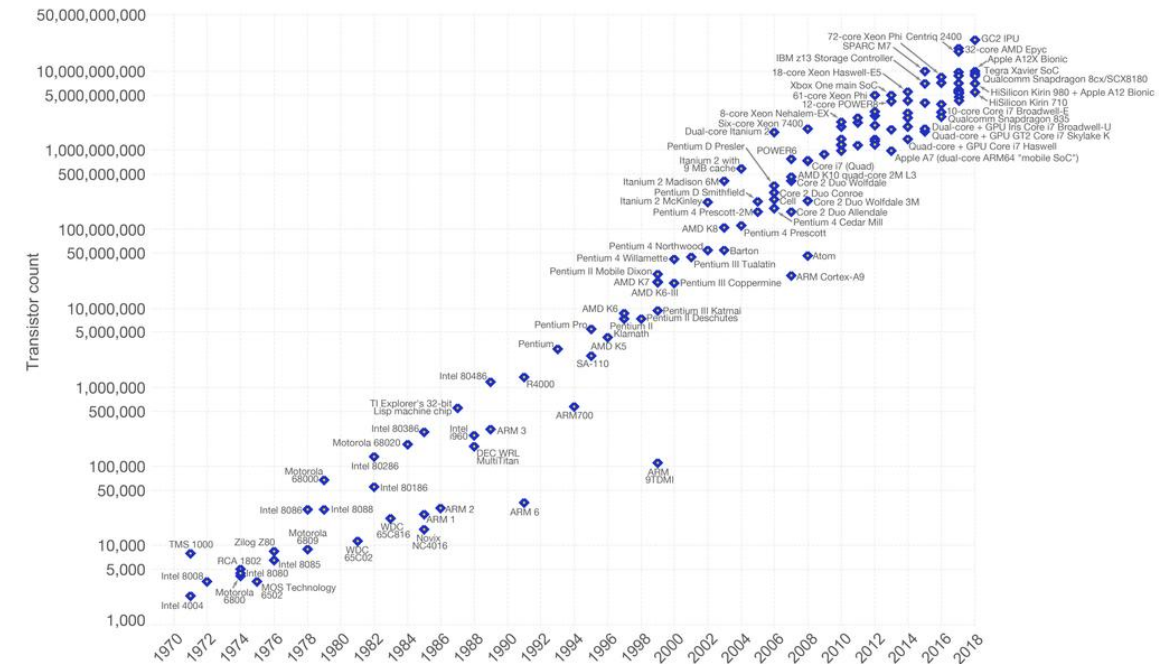


Kryder's Law (2005):
the density of information
doubles roughly every 13 months

Moore's Law – The number of transistors on integrated circuit chips (1971-2018)

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are linked to Moore's law.

Our World
in Data

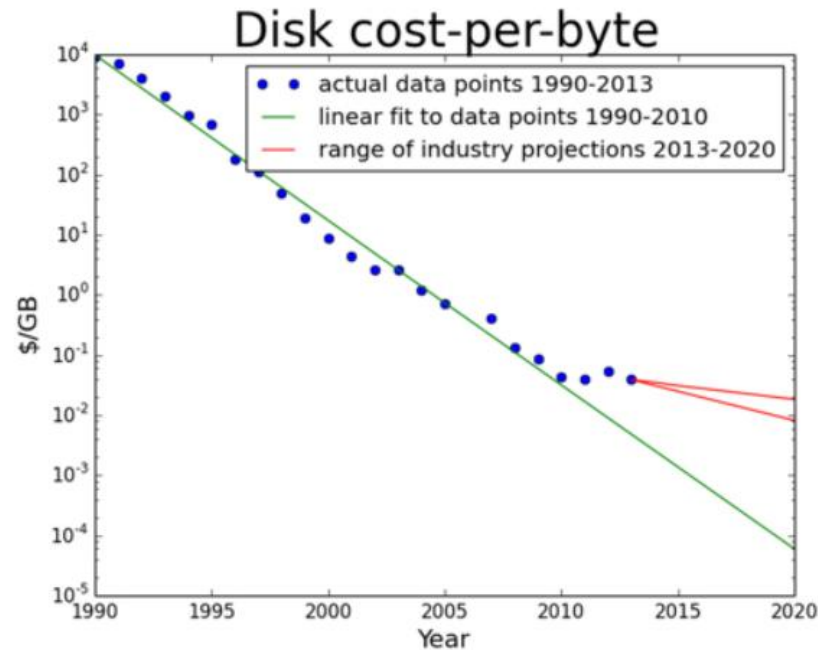


Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldInData.org. There you find more visualizations and research on this topic.

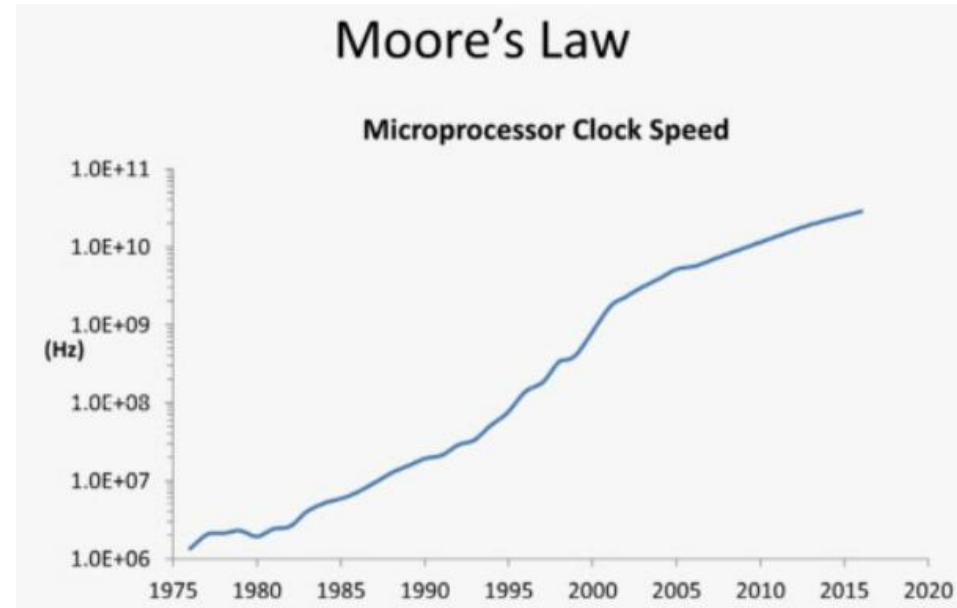
Licensed under CC-BY-SA by the author Max Roser.

Moore's law:
The processor speed doubles every 18 months
(combination of the effect of more transistors
and their being faster)

Behind the story



~~Kryder's Law:
the density of information
doubles roughly every 13 months~~



Moore's law:
The processor speed doubles every 18 months
(combination of the effect of more transistors
and their being faster)

Data growth

- ▶ **Gigabyte (1 000 000 000 bytes)**

- ▶ 1 Gigabyte: A pickup truck filled with paper OR A symphony in high-fidelity sound OR A movie at TV quality
- ▶ 2 Gigabytes: 20 meters of shelved books OR A stack of 9-track tapes

- ▶ **Terabyte (1 000 000 000 000 bytes)**

- ▶ All the X-ray films in a large technological hospital OR 50000 trees made into paper and printed
- ▶ 10 Terabytes: The printed collection of the US Library of Congress

- ▶ **Petabyte (1 000 000 000 000 000 bytes)**

- ▶ 2 Petabytes: All US academic research libraries
- ▶ 20 Petabytes: Production of hard-disk drives in 1995

- ▶ **Exabyte (1 000 000 000 000 000 000 bytes)**

- ▶ 5 Exabytes: All words ever spoken by human beings.

- ▶ **Zettabyte (1 000 000 000 000 000 000 000 bytes)**

- ▶ **Yottabyte (1 000 000 000 000 000 000 000 000 bytes)**

- ▶ **...what next?**



Parallel computing – paradigms

- ▶ Shared memory

- ▶ Partition the work

$F(w_p)$

share a

lock a[i]

process(w_p)

unlock

- ▶ Reduced efficiency because of waiting times

- ▶ Message passing:

- ▶ Partition the data:

$F(p)$

$a_p = a[p, \dots p+n/p-1]$

process (w)

exchange data (a_p)

- ▶ More scalable and easier to implement
 - ▶ Some algorithms are “embarrassing parallel” (do not need to exchange data)

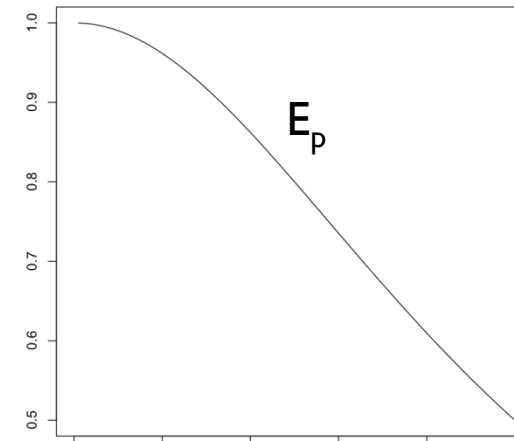
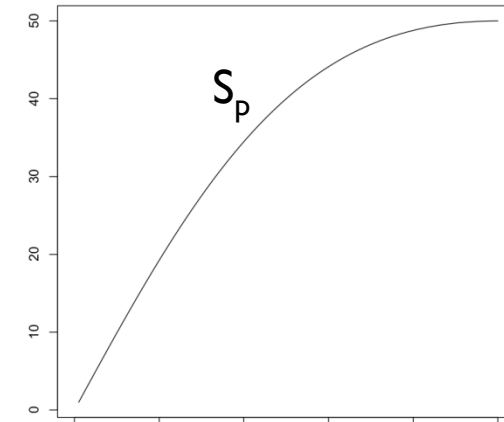


Parallel computing - efficiency

- ▶ **Speedup**: computing time with 1 processor (T_1) vs. computing time with p processors (T_p)

$$S_p = T_1 / T_p$$

- ▶ **Linear speedup** or **ideal speedup** is obtained when $S_p = p$
 - ▶ Scalability
- ▶ **Efficiency** is a performance metric:
$$E_p = S_p / p$$
 - ▶ estimates how well-utilized the processors are in solving the problem, compared to how much effort is wasted in communication and synchronization.
 - ▶ Algorithms with linear speedup and algorithms running on a single processor have an efficiency of 1
- ▶ **A scalable algorithm** is one for which E is an increasing function of n/p (volume of data vs. number of processors)



Map-Reduce

- ▶ A programming **model** based on the message passing paradigm, and an associated **infrastructure** for large scale computing on large data sets using commodity hardware

- ▶ Developed by Google and first presented in:

Jeffrey Dean and Sanjay Ghemawat 2004. [*MapReduce: simplified data processing on large clusters*](#). In Proceedings of the 6th conference on Symposium on Operating Systems Design & Implementation - Volume 6(OSDI'04), Vol. 6. USENIX Association, Berkeley, CA, USA, 10-10.

Google, nov 2008:

“It took six hours and two minutes to sort 1PB (10 trillion 100-byte records) on 4,000 computers. We're not aware of any other sorting experiment at this scale and are obviously very excited to be able to process so much data so quickly.

An interesting question came up while running experiments at such a scale: Where do you put 1PB of sorted data? We were writing it to 48,000 hard drives (we did not use the full capacity of these disks, though), and every time we ran our sort, at least one of our disks managed to break (this is not surprising at all given the duration of the test, the number of disks involved, and the expected lifetime of hard disks). To make sure we kept our sorted petabyte safe, we asked the Google File System to write three copies of each file to three different disks.”

<http://googleblog.blogspot.ro/2008/11/sorting-1pb-with-mapreduce.html>



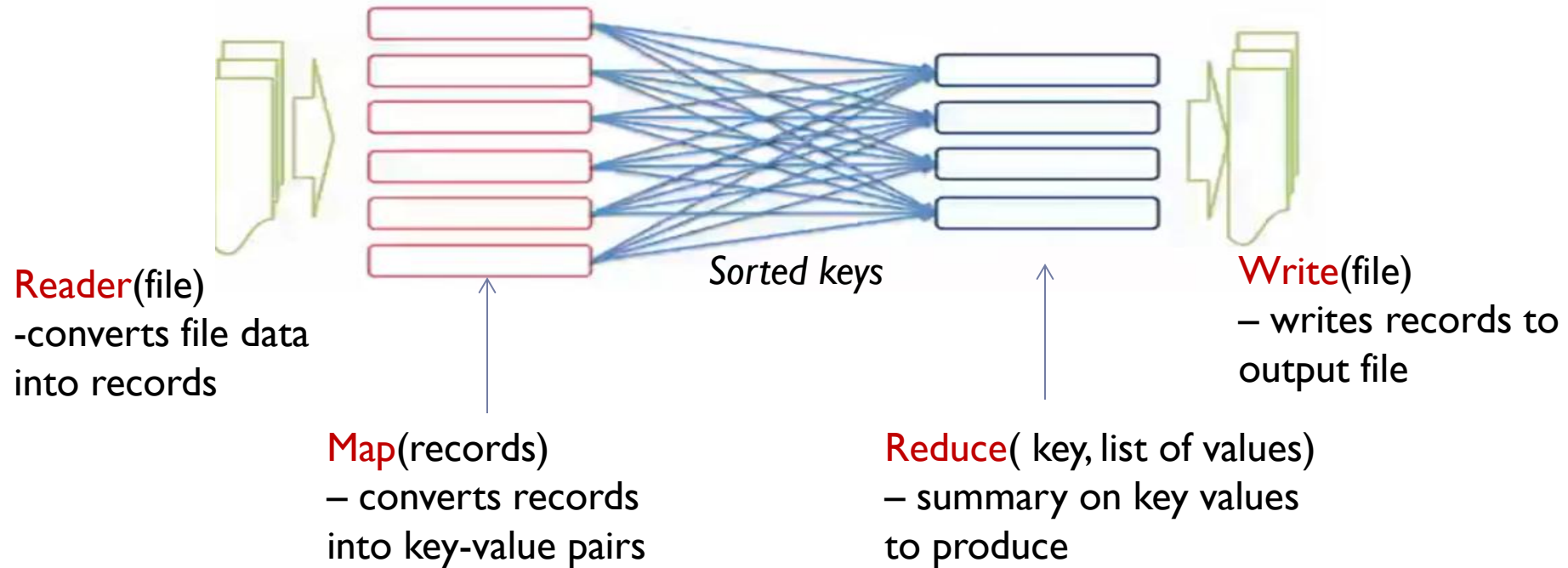
MapReduce

(Google, 2004)*

- ▶ “A *programming model and an associated implementation for processing and generating large data sets*”
 - ▶ Avoid data bottleneck
 - ▶ Avoid the burden of network programming
- ▶ Designed for batch processing over large datasets
- ▶ Favors scaling-out over scaling-up
- ▶ Roots in functional programming (*map* and *reduce/fold*):
 - ▶ The programmer specifies the two types of computation – **map** and **reduce** – and the execution framework should coordinate everything
 - ▶ Inputs and outputs are (key,value) pairs

Map Reduce – illustrated

Data is stored in files. Users provide functions:

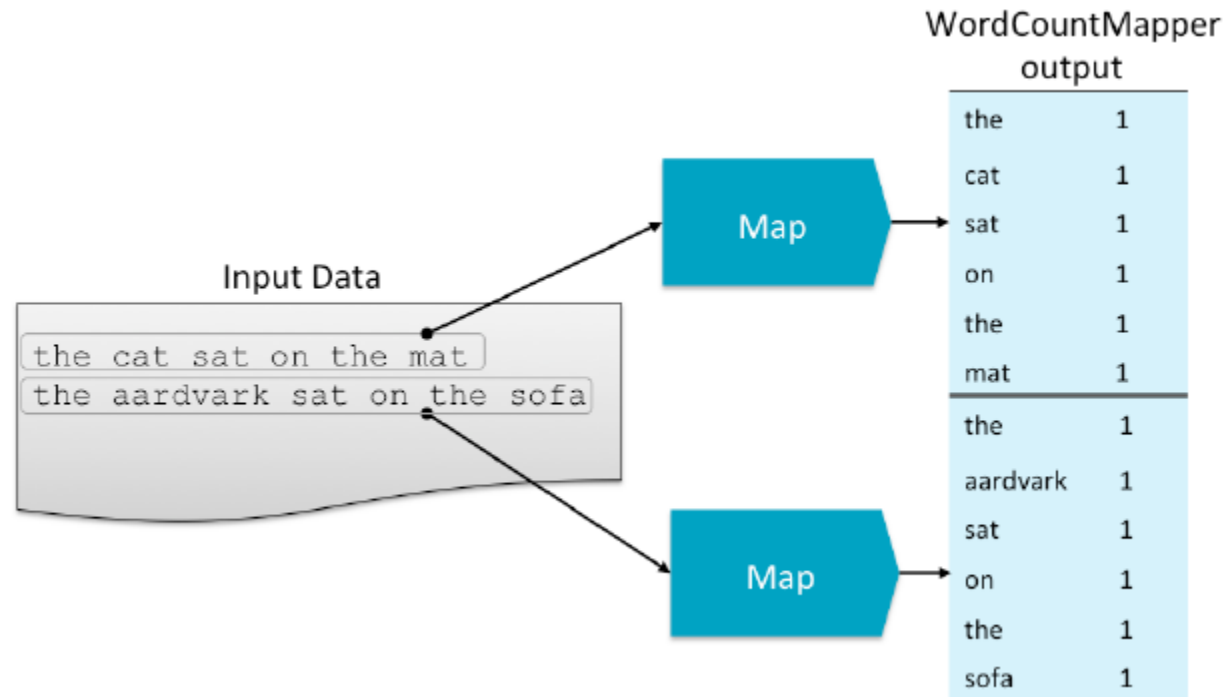


The map-reduce platform is responsible for *routing*



MapReduce example

Counting words - Mapper



MapReduce example

Counting words – Shuffle and sort

Mapper Output

the	1
cat	1
sat	1
on	1
the	1
mat	1
the	1
aardvark	1
sat	1
on	1
the	1
sofa	1

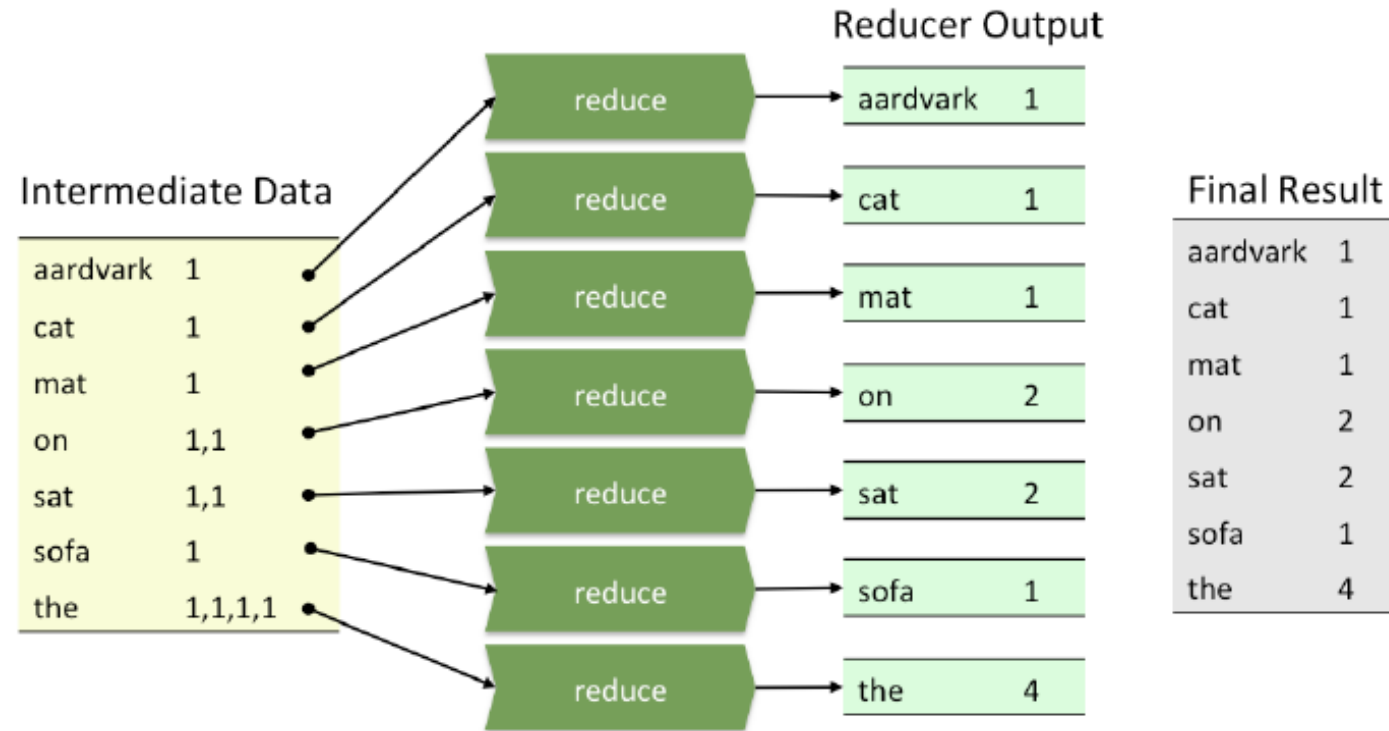


Intermediate Data

aardvark	1
cat	1
mat	1
on	1,1
sat	1,1
sofa	1
the	1,1,1,1

MapReduce example

Counting words – Reducer



References

- ▶ Eric Brewer's keynote on CAP: <http://www.cs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf>
- ▶ NoSQL database list: <http://nosql-database.org/>
- ▶ <https://www.mongodb.com/nosql-explained>

- ▶ *The Life of a Data Byte*: <https://queue.acm.org/detail.cfm?id=3419941>

